Nifty Volatility Forecaster: Concepts From Zero

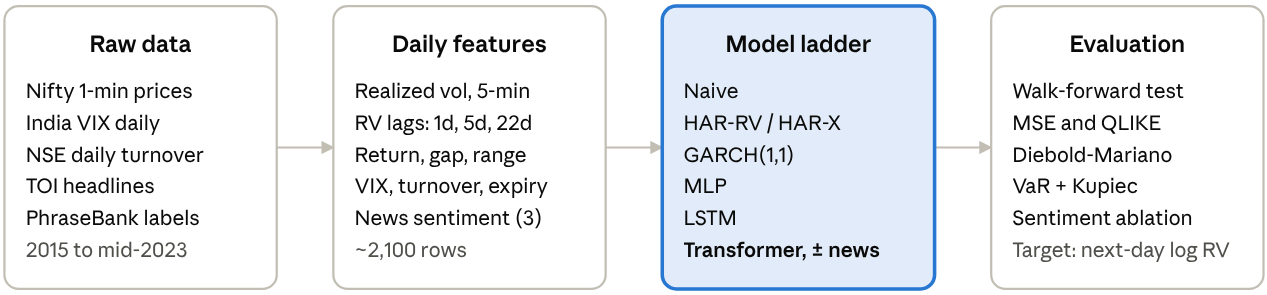
Oct 21, 2026 · @Shash

The project in one page

You're building a system that forecasts how much the Nifty 50 will move tomorrow (its volatility), not which way it moves. It learns from past intraday price moves, India VIX, trading turnover and the tone of Indian financial news headlines, and it compares six models, from a one-line rule up to a Transformer you write yourself in NumPy.

The design merges your two ideas. The volatility forecaster supplies the target and the classical baselines. The time-series Transformer and the news sentiment idea become the most advanced model on the ladder, with sentiment fed in as one more input.

Raw market and news data become one daily table that six models compete on



project pipeline · data to features to models to evaluation

Everything flows left to right once a day: the raw files become one feature row per trading day, every model forecasts the next day's log RV from the same rows, and all forecasts go through the same tests.

Why the target changed from price trend to volatility

The original Transformer idea predicted future price windows and trend direction. That framing has three problems, and fixing them makes the project stronger, not smaller.

1. Tomorrow's direction for a large index is close to a coin flip. A model that scores 52% on Nifty direction is normal, and with about 2,000 test days you can't tell 52% apart from luck. Examiners who know finance will ask about this.

2. Volatility clusters. Calm days follow calm days and wild days follow wild days, so tomorrow's volatility is genuinely predictable from today's. Your models can show a real, measurable win over a baseline.
3. News fits volatility better than direction. A shock headline (an RBI surprise, an election result, a war) reliably makes the market move more, even when nobody can say which way. So sentiment has a fair chance of helping here.

There's also a data reason. The free minute-level Nifty 50 index data has no volume column, because an index isn't traded directly (source). You'll use daily turnover of the index constituents instead, which NSE publishes (example).

You keep a direction prediction as an optional side task, reported honestly against the 50% line. That honesty is itself a good slide.

The model ladder

Each rung must beat the one below it to justify its complexity. If the Transformer loses to a linear regression, that's a valid result you report and explain.

Rung	Model	What it learns from	Why it's on the ladder
0	Naive	Today's volatility only	"Tomorrow looks like today." Anything that can't beat this is useless
1	HAR-RV	Volatility averaged over 1 day, 1 week, 1 month	The standard benchmark in volatility research; one linear regression
2	GARCH(1,1)	Daily returns	The classical statistical volatility model, fitted by maximum likelihood
3	MLP	All features, last few days flattened	First neural network; tests whether non-linearity helps at all
4	LSTM	All features as a 22-day sequence	Recurrent network; learns from order in time
5	Transformer	All features as a 22-day sequence, with and without sentiment	Attention decides which past days matter; the sentiment ablation is your headline experiment

What you can claim at the end

The deliverable is a table of forecast errors across all six models over about four years of out-of-sample days, a statistical test of whether the best model's win is real, and a risk check: if a trader set daily loss limits from your forecasts, how often would the market have broken them? That last part turns an accuracy number into money, which is what a panel remembers.

Data window: January 2015 to June 2023, where the minute-level price data and the news headline archive overlap (prices, headlines). That's roughly 2,100 trading days.

Finance basics from zero

Volatility is how much a price jumps around, and it's the quantity this whole project predicts. Everything else in this section builds up to it.

The Nifty 50

The Nifty 50 is a single number published by the National Stock Exchange (NSE) that tracks the combined value of 50 of India's largest listed companies, weighted by their size. When people say "the market fell 1% today" in India, they usually mean the Nifty. It trades from 9:15 am to 3:30 pm IST on weekdays, so each trading day has 375 minutes of prices.

You'll see four prices per time interval, called OHLC: the Open (first price), High, Low and Close (last price). A 5-minute OHLC bar summarises what the index did in those 5 minutes.

Why we use returns, not prices

A move from 20,000 to 20,200 and a move from 10,000 to 10,100 are both 1%, even though the point changes differ. Models need a scale-free measure, so we work with returns. We use the log return, the natural log of the ratio of consecutive prices:

```
r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)
```

For small moves, log return ≈ percentage return (a 1% rise gives r ≈ 0.00995). Log returns have one handy property: the return over a day equals the sum of the 5-minute log returns inside it.

Volatility

Volatility is the standard deviation of returns, a measure of how widely they spread around their average. A day where the Nifty swings ±0.2% is low volatility; a day with ±3% swings (like election results day in June 2024) is high.

The catch: you can't observe a single day's volatility directly. You only see one daily return. That's why we compute it from intraday data.

Realized volatility (RV): the thing we forecast

Split a trading day into 5-minute bars (75 bars). Compute each bar's log return, square them, and add them up. That sum is the day's realized variance; its square root is realized volatility.

```
RV_t = \sum_{i=1}^{M} r_{t,i}^2 \qquad \sigma^{\text{realized}}_t =
\sqrt{RV_t}
```

Here M is the number of intraday bars (75 for 5-minute bars) and r_{t,i} is the i-th return on day t. Squaring makes up and down moves count equally, which is what "how much it moved" means.

Why 5 minutes and not 1 minute? At 1-minute frequency, tiny price ticks bouncing between buyers and sellers (called microstructure noise) inflate the sum. Five minutes is the common compromise in research.

We forecast log(RV) rather than RV itself. RV is always positive and has occasional huge spikes; the log squashes those spikes so one crazy day doesn't dominate training, and it makes the error distribution closer to a bell curve.

Annualising, for intuition only: multiply daily volatility by √252 (about 15.9), since there are roughly 252 trading days a year. A daily volatility of 0.8% is about 12.7% a year.

Volatility clustering: why this is forecastable

Large moves tend to be followed by large moves, and small by small. Plot daily RV over ten years and you'll see calm stretches and stormy stretches (March 2020 stands out), not random scatter. Statistically, today's RV is strongly correlated with yesterday's and last week's. This persistence is what every model on the ladder exploits.

Compare that to returns: today's return tells you almost nothing about tomorrow's. That's the core reason the project predicts volatility.

The leverage effect

Volatility rises more after a fall than after a rise of the same size. Falling markets make people nervous, and nervous markets swing more. GARCH variants and your neural nets can learn this asymmetry; it's one reason negative news should matter more than positive news in your sentiment features.

India VIX

India VIX is NSE's "fear index". It's computed from Nifty option prices and represents the volatility traders expect over the next 30 days, quoted as an annual percentage (a VIX of 15 means traders expect about 15% annualised volatility). It's forward-looking where RV is backward-looking, so it's one of your strongest input features. An interesting result is whether your model beats VIX itself as a forecast.

Options expiry

Nifty options (contracts that pay out based on where the index ends up) expire on fixed weekdays. Expiry days often see unusual intraday swings as traders close positions. The expiry weekday has changed over the years, so you'll mark expiry days with a 0/1 feature built from the actual calendar, not a fixed weekday.

Value at Risk (VaR)

VaR answers: "what's the loss I shouldn't exceed on 99 out of 100 days?" If tomorrow's forecast volatility is σ and returns are roughly bell-shaped, the 99% one-day VaR is about 2.33 × σ. Banks and trading desks set limits this way. You'll use it to test whether your forecasts would have kept a risk manager safe.

The data and the features

Six public datasets feed one daily table: one row per trading day, about 2,100 rows, each row holding that day's features and the next day's log RV as the target.

Sources

Dataset	What's in it	Used for
Nifty 50 index minute data, 2015 to 2024 (Kaggle)	1-minute OHLC for the index; no volume	Realized volatility, returns, intraday range
India VIX daily history (NSE website)	Daily VIX open, high, low, close	Market's own forward-looking volatility
Nifty 50 daily data with shares traded and turnover (NSE, example on Kaggle)	Daily OHLC, shares traded, turnover in ₹ crore	Volume/activity feature
India News Headlines, 2001 to June 2023 (Times of India archive)	About 3.9 million headlines with date and category	News sentiment, filtered to business headlines
Financial PhraseBank (Malo et al., 2014)	4,840 financial news sentences labelled positive, neutral or negative	Training the sentiment classifier

Dataset	What's in it	Used for
Labelled Economic Times headlines (Kaggle)	400 Indian financial news items with sentiment labels	Checking the classifier works on Indian news, not just the training set

If the minute data has gaps (holidays, half days, the Diwali muhurat session), drop days with fewer than about 60 of the 75 five-minute bars rather than inventing prices.

Features (inputs) for day t

Group	Feature	How it's computed
Volatility history	log RV for day t	From 5-minute returns, as in the previous section
Volatility history	Weekly RV	Average RV over days t−4 to t
Volatility history	Monthly RV	Average RV over days t−21 to t
Range	Parkinson volatility	From the day's high and low: $(\ln H - \ln L)^2 / (4 \ln 2)$
Returns	Daily return	ln(close_t / close_t−1)
Returns	Negative return	min(daily return, 0); lets models see the leverage effect
Returns	Overnight gap	ln(open_t / close_t−1); news that broke while the market was shut
Expectations	India VIX (log)	Close value on day t
Activity	Turnover change	log of turnover_t divided by its 20-day average
Calendar	Expiry day flag	1 if day t+1 is an options expiry day, else 0
Calendar	Day of week	One-hot (5 columns of 0/1)
News	Mean headline sentiment	Average classifier score over business headlines on day t

Group	Feature	How it's computed
News	Negative share	Fraction of day t's headlines classified negative
News	Headline count (log)	Unusual news volume is itself a signal

Target: log RV for day t+1. The expiry flag looks at t+1, which is fine because the expiry calendar is known in advance.

Scaling

Neural networks train badly when one input ranges 0 to 1 and another ranges 10,000 to 30,000. Standardise each feature: subtract its mean and divide by its standard deviation, so each has mean 0 and spread 1. Compute that mean and standard deviation on the training period only, then apply them unchanged to later data.

The leakage trap

Leakage means the model secretly sees information from the future during training, so its test score looks great and means nothing. Time-series projects are full of ways to do this by accident:

- Shuffling days randomly before splitting into train and test. Never do this; always split by date.
- Scaling with the mean and standard deviation of the whole dataset (the future leaks into the past).
- Using headlines from day t+1 to predict day t+1. A headline published at 6 pm on day t is fine; one published at 11 am on day t+1 is not. The archive only gives dates, not times, so use headlines from day t only.
- Filling missing values with a backward fill, which copies future values into the past.

A useful sanity check: if any model beats the naive baseline by a huge margin (say a 60% error cut), suspect leakage before celebrating.

The math toolkit

You need four ideas: arrays of numbers, derivatives, gradient descent, and maximum likelihood. Every model in the project is some combination of these.

Vectors, matrices and shapes

A vector is a list of numbers, such as one day's 14 features. A matrix is a grid, such as 2,100 days × 14 features. In NumPy these are arrays, and the single most common bug in from-scratch deep learning is a shape mismatch, so write the shape next to every array in comments.

Matrix multiplication is how a layer combines inputs. If X has shape (batch, inputs) and W has shape (inputs, outputs), then X @ W has shape (batch, outputs): each output is a weighted sum of all the inputs.

```
(XW)_{ij} = \sum_k X_{ik} W_{kj}
```

The transpose W^T flips rows and columns. It shows up constantly in backpropagation.

Derivatives and the chain rule

A derivative tells you how much an output changes when you nudge an input a tiny bit. If loss L depends on weight w, then dL/dw > 0 means increasing w increases the loss, so you should decrease w.

The chain rule handles functions inside functions. If L depends on y, and y depends on w:

```
\frac{dL}{dw} = \frac{dL}{dy} \cdot \frac{dy}{dw}
```

A neural network is a long chain of simple functions, so its gradient is a long chain-rule product. Backpropagation is just computing that product efficiently from the output back to the input.

A partial derivative ðL/ðw is the same idea when L depends on many variables: change w, hold everything else fixed. The gradient ∇L is the vector of all partial derivatives, one per parameter.

Checking gradients numerically

When you write backprop by hand, you'll make mistakes. The fix is a gradient check: nudge one weight by a tiny ε both ways and see how the loss changes.

```
\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w-
\varepsilon)}{2\varepsilon}
```

With ε = 1e-5, the relative difference between this and your analytic gradient should be below about 1e-6 (use float64 for the check). Run this on every layer before training anything. It's the single most valuable habit in the project.

Gradient descent

Training means finding weights that make the loss small. Gradient descent repeats one step: move every weight a little in the direction that lowers the loss.

```
w \leftarrow w - \eta \, \nabla_w L
```

η (eta) is the learning rate, the step size. Too big and training explodes; too small and it crawls. Typical starting values are 1e-3 for Adam.

You don't compute the loss over all 2,100 days for every step. Instead you take a mini-batch (say 32 random training windows), compute the gradient on those, and step. That's stochastic gradient descent. One pass over all training data is an epoch.

Adam

Adam is the optimiser you'll actually use. It keeps two running averages per weight: the average gradient (momentum, m) and the average squared gradient (v). Weights with consistently large gradients take smaller steps; weights with small, noisy gradients take relatively bigger ones.

```
m \leftarrow \beta_1 m + (1-\beta_1) g \qquad v \leftarrow \beta_2 v + (1-
\beta_2) g^2
```

```
\hat m = \frac{m}{1-\beta_1^t} \qquad \hat v = \frac{v}{1-\beta_2^t} \qquad w
\leftarrow w - \eta \frac{\hat m}{\sqrt{\hat v} + \epsilon}
```

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, ε = 1e-8. The hat versions correct for m and v starting at zero; t is the step count. It's about 15 lines of NumPy.

Maximum likelihood estimation (MLE)

GARCH isn't trained with a loss like mean squared error. It's fitted by maximum likelihood: pick the parameters under which the data you actually observed would have been most probable.

If returns follow a normal (bell-curve) distribution with mean 0 and variance σ²_t, the probability density of one return is:

```
f(r_t) = \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\left(-\frac{r_t^2}
{2\sigma_t^2}\right)
```

Multiplying thousands of small probabilities underflows to zero, so we take logs and add instead. Maximising the log-likelihood is the same as minimising its negative:

```
-\log \mathcal{L} = \frac{1}{2}\sum_t \left(\log 2\pi + \log \sigma_t^2 +
\frac{r_t^2}{\sigma_t^2}\right)
```

You'll minimise this with gradient descent (or a simple grid search followed by gradient descent), which reuses the same machinery as your neural networks.

Ordinary least squares (OLS)

HAR-RV is a linear regression, which has an exact answer with no training loop. Stack the features into X (with a column of 1s for the intercept) and the targets into y:

```
\hat\beta = (X^\top X)^{-1} X^\top y
```

In NumPy, use np.linalg.solve(X.T @ X, X.T @ y) rather than computing the inverse directly; it's more numerically stable. np.linalg is plain linear algebra, not an ML library, but confirm with your professor.

Classical baselines: HAR-RV and GARCH

These two models are the bar your neural networks must clear. HAR-RV in particular is hard to beat, and published research often finds deep models only match it.

HAR-RV (Heterogeneous Autoregressive model of Realized Volatility)

Fulvio Corsi proposed HAR-RV in 2009. The idea: different traders care about different horizons. Day traders react to yesterday, swing traders to last week, funds to last month. So predict tomorrow's volatility from three averages:

```
\log RV_{t+1} = \beta_0 + \beta_d \log RV_t + \beta_w \log RV^{(w)}_t +
\beta_m \log RV^{(m)}_t + \varepsilon_{t+1}
```

$RV^{(w)}$ is the average RV over the last 5 days and $RV^{(m)}$ over the last 22. That's four numbers to learn, fitted exactly with OLS. It's a 20-line implementation and usually explains a large share of the day-to-day variation in log RV.

You'll also build HAR-X: the same regression with extra columns for VIX, the negative return, and sentiment. If HAR-X with sentiment beats plain HAR, you've shown sentiment helps even before any neural network.

GARCH(1,1) (Generalised Autoregressive Conditional Heteroskedasticity)

Robert Engle's ARCH model (1982) and Tim Bollerslev's generalisation (1986) model tomorrow's variance as a blend of three things: a long-run baseline, today's surprise, and today's variance.

```
\sigma^2_{t+1} = \omega + \alpha \, r_t^2 + \beta \, \sigma^2_t
```

Read it as: tomorrow's variance = a constant + a reaction to today's squared return (α) + memory of today's variance (β). The sum α + β is the persistence. For equity indices it's typically close to 1 (around 0.95 to 0.99), meaning shocks fade slowly. The long-run variance the model drifts back to is ω / (1 − α − β).

Constraints keep it sensible: ω > 0, α ⩾ 0, β ⩾ 0, and α + β < 1. Gradient descent doesn't respect constraints, so optimise unconstrained numbers and transform them:

- ω = exp(a), which is always positive
- persistence p = sigmoid(b), which is between 0 and 1
- α = p × sigmoid(c) and β = p − α, which splits the persistence between the two

Fitting: run the recursion through the training data starting from σ^2_0 = the sample variance, compute the negative log-likelihood from the math toolkit section, and minimise it over (a, b, c). The gradient through the recursion is a small backprop-through-time exercise, good practice for the LSTM. If that's painful, a grid search over p and the α share followed by fine-tuning works fine for three parameters.

One mismatch to handle: RV covers 9:15 to 3:30 only, while close-to-close returns include the overnight gap. Fit GARCH on open-to-close returns so both measure the same window. Then compare log of the GARCH forecast with log RV.

GJR-GARCH is a one-term extension that captures the leverage effect: add γ × r^2_t × 1[r_t < 0], so falls raise variance more than rises. It's an easy stretch goal.

Neural networks from scratch

A neural network is a stack of layers, each doing a matrix multiply followed by a simple non-linear function. Training adjusts the weights so the output gets closer to the target. Writing it yourself means writing two functions per layer: forward (compute the output) and backward (compute gradients).

One layer: the linear (dense) layer

```
Y = XW + b
```

X is (batch, in), W is (in, out), b is (out,). Each output is a weighted sum of inputs plus a bias. Given the gradient of the loss with respect to the output, dY, the backward pass is:

```
\frac{\partial L}{\partial W} = X^\top dY \qquad \frac{\partial L}{\partial b} = \sum_{\text{batch}} dY \qquad \frac{\partial L}{\partial X} = dY \, W^\top
```

The last one, dX, is handed to the layer before. That handing back, layer by layer, is backpropagation. Save X during the forward pass because backward needs it.

Activations: where non-linearity comes from

Stacking linear layers without anything between them collapses into one linear layer. An activation function between them lets the network bend.

Activation	Formula	Derivative	Where you'll use it
ReLU	max(0, x)	1 if x > 0, else 0	MLP hidden layers
GELU	x × Φ(x), usually the tanh approximation	Longer; derive it once and gradient-check	Transformer feed-forward block
Sigmoid	$1 / (1 + e^{-x})$	s(1 − s)	LSTM gates, sentiment classifier output
Tanh	$(e^{x} - e^{-x}) / (e^{x} + e^{-x})$	$1 - \tanh^2$	LSTM cell
Softmax	$e^{xi} / \Sigma\, e^{xj}$	Jacobian; see the Transformer section	Attention weights, 3-class sentiment

For softmax, subtract the row maximum before exponentiating. The answer is identical and it stops e^{1000} from overflowing.

Loss functions

The loss is one number measuring how wrong the predictions are. For the volatility forecast, you'll train on mean squared error of log RV:

```
L = \frac{1}{N}\sum_{i=1}^{N} (\hat y_i - y_i)^2 \qquad \frac{\partial L}{\partial \hat y_i} = \frac{2}{N}(\hat y_i - y_i)
```

For the sentiment classifier you'll use cross-entropy, which punishes confident wrong answers heavily. With softmax outputs p and a one-hot true label, its gradient with respect to the pre-softmax scores is simply p − label, one of the nicest results in deep learning.

Weight initialisation

Start with weights that are too big and signals explode through layers; too small and they vanish. Use He initialisation for ReLU layers (normal noise with standard deviation √(2 / inputs)) and Xavier (Glorot) for tanh and sigmoid (standard deviation √(1 / inputs)). Biases start at zero, except the LSTM forget gate bias, which starts at 1.

Regularisation: fighting overfitting

With only about 1,400 training days, overfitting is the main risk. The network memorises training noise and fails on new days. Four tools, all simple to write:

- Weight decay (L2): add λ × Σw² to the loss, which pulls weights toward zero. In AdamW form you subtract η × λ × w from each weight directly at each step.
- Dropout: during training, randomly zero a fraction of activations (say 10%) and scale the rest up by 1 / (1 − rate). At test time, do nothing. Store the mask for the backward pass.
- Early stopping: track the validation loss after every epoch and keep the weights from the best epoch. Stop when it hasn't improved for 15 or so epochs.
- Small models: a Transformer with 30,000 parameters fitted to 1,400 rows will overfit. Keep parameter counts in the low thousands to low tens of thousands.

Gradient clipping

If the overall size (norm) of the gradient exceeds a threshold like 1.0, scale it down to that threshold. One bad batch can't then throw the weights somewhere useless. You'll need this for the LSTM.

How the code should be organised

Give every layer the same three methods: forward(x), backward(dout) and a list of (parameter, gradient) pairs the optimiser reads. Then an MLP, LSTM and Transformer are just different lists of the same building blocks, and one training loop serves all of them. The F1 pitstop network you built is a head start: reuse its dense layer and optimiser if they pass a gradient check.

LSTM: learning from sequences

An LSTM reads the last 22 trading days one at a time, carrying a memory forward, and predicts tomorrow's log RV from its final memory. Unlike the MLP, which sees all 22 days flattened into one long vector, it knows day 22 came after day 21.

Plain recurrent networks and why they fail

A basic recurrent network (RNN) keeps a hidden state h and updates it each step: h_t = tanh(W x_t + U h_{t−1} + b). During training, the gradient for an early day passes through the same multiplication by U once per step. Multiply by something smaller than 1 twenty times and the gradient vanishes, so the network can't learn long-range patterns. Bigger than 1 and it explodes.

The LSTM fix: a protected memory with gates

Long Short-Term Memory (Hochreiter and Schmidhuber, 1997) adds a cell state c, a memory lane that's updated by addition rather than repeated multiplication. Three gates, each a sigmoid output between 0 and 1, control it:

```
f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)
```

```
\tilde c_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \qquad c_t = f_t \odot c_{t-1} + i_t \odot \tilde c_t \qquad h_t = o_t \odot \tanh(c_t)
```

⊙ means element-wise multiplication. In plain words:

1. The forget gate f decides how much of the old memory to keep. In a calm market it might learn to keep a lot.
2. The input gate i decides how much of the new candidate memory c̃ to write. A large overnight gap or a cluster of negative headlines might open it wide.
3. The output gate o decides how much of the memory to expose as h.

Implementation trick: compute all four pre-activations in one matrix multiply with a stacked weight of shape (inputs + hidden, 4 × hidden), then slice. It's faster and the backward pass is cleaner.

Backpropagation through time (BPTT)

To train, unroll the LSTM across all 22 steps, run forward while saving every gate value, c and h, then walk backward from step 22 to step 1. At each step you receive two incoming gradients, dh from the step after and dc from the step after, and you produce gradients for the weights (summed across all steps, since every step shares the same weights) plus dh and dc for the step before.

The key backward line is how the cell gradient flows:

```
dc_{t-1} = dc_t \odot f_t
```

Because f can stay near 1, the gradient survives many steps. That's the whole reason LSTMs work. Gradient-check the full unrolled LSTM on a tiny case (2 inputs, 3 hidden units, 4 steps) before trusting it.

Sizes for this project

One LSTM layer with 16 to 32 hidden units, followed by a dense layer to one output. With 14 inputs and 32 hidden units that's about 6,000 parameters, appropriate for ~1,400 training rows. Use gradient clipping at a norm of 1.0.

The time-series Transformer

The Transformer looks at all 22 days at once and learns, for each day, how much attention to pay to every other day. Where an LSTM passes memory along step by step, attention lets day 22 look straight back at the crash on day 3 without the signal fading.

How it differs from a language Transformer

In ChatGPT-style models, each input position is a word, turned into a vector by looking it up in an embedding table. Here each position is a trading day with 14 numbers (17 with sentiment). Replace the lookup table with a linear layer that projects each day's feature vector into a d-dimensional vector (d = 32). Everything after that is the standard architecture from Vaswani et al., "Attention Is All You Need" (2017).

You also don't need a decoder or text generation. You need an encoder that reads the window and a small head that outputs one number: tomorrow's log RV.

Positional encoding

Attention on its own is order-blind: shuffle the 22 days and it computes the same thing. So add a position vector to each day's embedding. Two options:

- Sinusoidal (fixed): sine and cosine waves of different frequencies, as in the original paper. No parameters.
- Learned: a (22, d) parameter matrix trained like any weight. Simple, and fine for a fixed window of 22.

Try learned first; it's one more parameter array and one line of backward.

Self-attention, step by step

From the embedded window H (shape 22 × d), compute three matrices with three linear layers: queries Q = HW_Q, keys K = HW_K and values V = HW_V. Think of it like a search: each day's query asks "what am I looking for?", each day's key says "here's what I contain", and the value is what gets passed along if there's a match.

```
\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}
{\sqrt{d_k}}\right)V
```

1. QK^T is a 22 × 22 grid of scores: how relevant is day j to day i.
2. Dividing by $\sqrt{d_k}$ keeps scores from growing with dimension, which would push softmax into near-0/1 outputs with tiny gradients.
3. Softmax along each row turns scores into weights that sum to 1.
4. Multiplying by V gives each day a weighted blend of all days' values.

The 22 × 22 weight grid is something you can plot. Averaged over the test set, it shows which past days the model relies on, for example whether it consistently looks at the most recent day and at the previous expiry day. That plot is your interpretability slide.

Causal masks: do you need one?

In language models, a causal mask stops a word from attending to later words. Here all 22 days are in the past relative to the target, so letting day 5 attend to day 9 leaks nothing. No mask is needed. The leakage risk lives in how you build the windows and targets, not in the attention.

Multi-head attention

Instead of one attention with d = 32, run 2 to 4 heads with d_k = 32 / heads each, in parallel, concatenate the results and pass them through one more linear layer W_O. Different heads can specialise: one on the last few days, one on spikes.

In NumPy, reshape (batch, 22, d) into (batch, heads, 22, d_k) and use np.matmul or the @ operator, which handles the extra batch dimensions.

The encoder block

One block, in the pre-norm arrangement (more stable for small from-scratch models):

```
Z = H + \text{MultiHead}(\text{LayerNorm}(H)) \qquad H' = Z + \text{FFN}
(\text{LayerNorm}(Z))
```

The FFN is dense (d → 2d), GELU, dense (2d → d), applied to each day separately. The "H +" parts are residual connections: the block learns a correction to its input rather than a full replacement, which keeps gradients flowing.

Layer norm normalises each day's d-vector to mean 0 and variance 1, then applies learned scale γ and shift β:

```
\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} +
\beta
```

The output head

After 1 or 2 blocks, take the vector for the last day (day 22) or the average over all days, and pass it through a dense layer to one number. Try both; averaging is usually more stable on small data.

The backward passes you'll write

Three are new compared with the MLP and LSTM. Gradient-check each one alone.

1. Softmax backward. If $P = \text{softmax}(S)$ and dP is the incoming gradient, then $dS = P \odot (dP - \text{rowsum}(dP \odot P))$. One line in NumPy with keepdims=True.
2. Attention backward. With $A = PV$: $dV = P^T dA$ and $dP = dA\, V^T$; then softmax backward gives dS; and with $S = QK^T/\sqrt{d_k}$, $dQ = dS\, K / \sqrt{d_k}$ and $dK = dS^T Q / \sqrt{d_k}$.
3. Layer norm backward. With $\hat{x}$ the normalised input and $N = d$: $dx = (1/(N\,\sigma)) \times (N\, d\hat{x} - \Sigma d\hat{x} - \hat{x}\, \Sigma(d\hat{x} \odot \hat{x}))$, where $d\hat{x} = dy \odot \gamma$ and $\sigma = \sqrt{(\sigma^2 + \varepsilon)}$.

Sizes for this project

d = 32, 2 heads, 1 encoder block, FFN width 64, dropout 0.1. That's roughly 9,000 parameters, similar to the LSTM, which makes the comparison fair. Add a second block only if validation loss says it helps.

Where sentiment comes in

The three news features are appended to each day's feature vector before the input projection (early fusion). The ablation is simple: train the identical model with and without those three columns, five times each with different random seeds, and compare average test error. Five seeds matter because small networks vary a lot run to run, and you need to show the difference is bigger than that noise.

News sentiment from scratch

You'll train a small text classifier on labelled financial sentences, run it over every business headline in the Times of India archive, and boil each day down to three numbers. No pretrained language model is allowed, so the classifier is a bag-of-words model you write yourself.

Step 1: turn text into numbers

1. Lowercase, strip punctuation, split on spaces. That's tokenisation.
2. Build a vocabulary: every word that appears at least 3 times in the training sentences, plus common two-word phrases (bigrams) like "profit falls" or "rate cut". Bigrams matter in finance because "not good" and "good" differ.
3. Represent each sentence as a vector of length |vocabulary|, with TF-IDF weights.

TF-IDF (term frequency × inverse document frequency) gives a word a high score if it appears in this sentence but is rare across sentences. "the" scores near zero; "default" scores high.

```
\text{tfidf}(w, s) = \text{tf}(w, s) \times \left(\log\frac{1 + N}{1 +
\text{df}(w)} + 1\right)
```

N is the number of training sentences and df(w) is how many contain word w. Compute df on training sentences only. Store the vectors as dense NumPy arrays if the vocabulary is under about 10,000 words; Financial PhraseBank is small enough.

Step 2: the classifier

Multinomial logistic regression (softmax regression): one dense layer from the TF-IDF vector to 3 scores (negative, neutral, positive), then softmax, trained with cross-entropy and L2 weight decay. It's your dense layer and softmax from earlier, reused.

Financial sentiment differs from everyday sentiment. "Liability", "tax" and "cost" sound negative in plain English but are often neutral in finance. That's why you train on Financial PhraseBank (4,840 sentences labelled from an investor's point of view by finance-literate annotators) rather than movie reviews (dataset card).

Optionally compare against a Naive Bayes classifier (counting word frequencies per class, about 20 lines). If logistic regression doesn't beat it, something's wrong.

Step 3: check it on Indian news

PhraseBank sentences come from international news. Indian headlines use different vocabulary ("Sensex", "FII outflow", "repo rate"). Test the trained classifier on the 400 labelled Economic Times items before trusting it on the archive (dataset). Expect lower accuracy than on PhraseBank, and report both numbers.

Step 4: from headlines to a daily signal

Filter the archive to business and market categories (look at the headline_category values; anything containing "business" is a start). For each trading day, compute:

- Mean sentiment score: positive probability minus negative probability, averaged over that day's headlines
- Negative share: fraction of headlines where negative is the top class
- Log headline count

Weekends and holidays have news but no trading. Roll those headlines into the next trading day, because that's when the market reacts to them. Remember the rule from the data section: day t's features use only headlines dated day t or earlier.

Honest expectations

The signal will be noisy. A headline classifier trained on 4,840 sentences won't understand sarcasm or context, and most business headlines don't move the index. The research question is whether the noisy signal still adds anything on top of VIX and past volatility. A clear "no" is a valid finding, and you should present it confidently if that's what the data says.

Evaluation: proving a model is actually better

The project's credibility rests here. You'll compare all models on the same unseen days with two error measures, a significance test, and a risk backtest.

Splitting by time

Use a chronological split, never a random one:

Period	Approximate dates	Days	Purpose
Train	Jan 2015 to Dec 2019	~1,230	Fit weights
Validation	Jan 2020 to Dec 2020	~250	Choose hyperparameters, early stopping (includes the COVID crash, a deliberate stress test)
Test	Jan 2021 to Jun 2023	~620	Touched once, at the end, for the final numbers

Then add walk-forward evaluation for the final comparison: train on everything up to a date, forecast the next 3 months, slide forward 3 months, retrain, repeat. That mimics how a real desk would use the model and shows whether performance holds across market regimes. Refitting HAR and GARCH each step is cheap; for the neural networks, retraining every quarter is enough.

Error measure 1: MSE on log RV

The training loss, reported on test days. Easy to understand and symmetric.

Error measure 2: QLIKE

The standard loss in volatility forecasting research, because it ranks forecasts correctly even when the "true" volatility is measured with noise (Patton, 2011). With RV the realised variance and h the forecast variance:

```
\text{QLIKE} = \frac{RV}{h} - \log\frac{RV}{h} - 1
```

It's zero for a perfect forecast and punishes under-prediction more than over-prediction. That asymmetry matches what risk managers fear: being caught out by more volatility than expected. Convert your log RV forecasts back with exp() before computing it.

Is the win real? The Diebold-Mariano test

If the Transformer's average QLIKE is 3% lower than HAR's, that might be luck. The Diebold-Mariano test (1995) checks it. For each test day, compute the loss difference d_t = loss_HAR − loss_Transformer. If the models are equally good, d_t averages zero.

```
DM = \frac{\bar d}{\sqrt{\hat V(\bar d)}}
```

Compare DM to a standard normal: |DM| > 1.96 means the difference is significant at the 5% level. Because daily losses are correlated over time, estimate the variance with a Newey-West correction (a weighted sum of autocovariances over a few lags) rather than the plain sample variance. It's about 15 lines of NumPy, and "statistically significant improvement" lands far better than "lower error".

Turning forecasts into money: a VaR backtest

For each test day, set the 99% VaR as 2.33 × forecast daily volatility. Count the days where the actual open-to-close loss exceeded it (a breach). A good model breaches on about 1% of days, roughly 6 in 620.

The Kupiec test (1995) checks whether the breach count is consistent with 1%. With n test days, x breaches and p = 0.01:

```
LR = -2\ln\left[(1-p)^{n-x}p^{x}\right] + 2\ln\left[\left(1-\tfrac{x}
{n}\right)^{n-x}\left(\tfrac{x}{n}\right)^{x}\right]
```

If LR exceeds 3.84 (the 5% critical value of a chi-squared distribution with 1 degree of freedom), the model's risk estimates are rejected. Too many breaches means it underestimated risk; far too few means it was so cautious that capital was wasted. Report

the breach count for every model in one table; this is the slide a finance-literate examiner will remember.

Ablations: what actually helped

An ablation removes one ingredient and measures the damage. Run at least these, each averaged over 5 seeds:

- Transformer with vs without the three sentiment features (the headline experiment)
- Transformer with vs without India VIX
- LSTM vs Transformer at a similar parameter count
- HAR vs HAR-X with sentiment (does sentiment help even a linear model?)

The optional direction task

If you also predict whether tomorrow's return is up or down, report accuracy next to the 50% coin-flip line and the share of up days in the test set (often around 53%, so "always say up" is the real baseline). Use a binomial test to check whether your accuracy beats that. Expect it not to, and say why: it reinforces the reason the project forecasts volatility.

Glossary

Term	Meaning in one line
Ablation	Removing one input or component to measure how much it contributed
Adam	Optimiser that adapts each weight's step size using running averages of gradients
Attention	Weighted mixing of positions, where the weights come from query-key similarity
Backpropagation	Computing all weight gradients by applying the chain rule from the loss backward
BPTT	Backpropagation through time; backprop across the unrolled steps of a recurrent network
Cross-entropy	Classification loss; negative log of the probability given to the true class

Term	Meaning in one line
Diebold-Mariano test	Tests whether two forecasts' average losses really differ
Dropout	Randomly zeroing activations during training to reduce overfitting
Early stopping	Keeping the weights from the epoch with the best validation loss
Epoch	One full pass over the training data
GARCH(1,1)	Model where tomorrow's variance depends on today's squared return and today's variance
Gradient check	Comparing hand-derived gradients with numerical finite differences
HAR-RV	Linear regression of tomorrow's volatility on daily, weekly and monthly averages
India VIX	NSE's index of expected Nifty volatility over the next 30 days, from option prices
Kupiec test	Tests whether VaR breaches happen at the promised rate
Layer norm	Rescaling each position's vector to mean 0 and variance 1, then a learned scale and shift
Leakage	Future information accidentally available during training, inflating test scores
Leverage effect	Volatility rising more after price falls than after equal rises
Log return	ln(P_t / P_t−1); scale-free price change
LSTM	Recurrent network with gated memory that keeps gradients alive across many steps
MLE	Choosing parameters that make the observed data most probable
Nifty 50	NSE index of 50 large Indian companies
OHLC	Open, high, low and close prices for an interval

Term	Meaning in one line
OLS	Ordinary least squares; exact solution for linear regression
Positional encoding	Vectors added to inputs so attention knows the order of positions
QLIKE	Volatility forecast loss that stays reliable when true volatility is measured noisily
Realized volatility (RV)	Square root of the sum of squared intraday returns for a day
Residual connection	Adding a block's input to its output so it learns a correction
Softmax	Turns a vector of scores into probabilities that sum to 1
TF-IDF	Word weight that's high when a word is frequent in a text but rare overall
VaR	Loss threshold expected to be exceeded on only a chosen share of days, such as 1%
Volatility clustering	Large moves following large moves, small following small
Walk-forward evaluation	Repeatedly training on the past and testing on the next period, sliding forward